

Introduction to HTML

Introduction to HTML

Welcome to the world of code! Last year, millions of learners from our community started with HTML. Why? HTML is the skeleton of all web pages. It's often the first language learned by developers, marketers, and designers and is core to front-end development work. If this is your first time touching code, we're excited for what you're about to create.

HTML provides structure to the content appearing on a website, such as images, text, or videos. Right-click on any page on the internet, choose "Inspect," and you'll see HTML in a panel of your screen.

HTML stands for HyperText Markup Language:

=>A *markup* language is a computer language that defines the structure and presentation of raw text.

=>In HTML, the computer can interpret *raw text* that is wrapped in HTML elements.

=>*HyperText* is text displayed on a computer or device that provides access to other text through links, also known as *hyperlinks*. You probably clicked on a couple of hyperlinks on your way to this Codecademy course.

Learning HTML is the first step in creating websites, but even a bit of knowledge can help you inject code snippets into newsletter, blog or website templates. As you continue learning, you can layer HTML with CSS and JavaScript to create visually compelling and dynamic websites. But for now, we're going to focus on how to add and modify basic content on a page, like text, images, and videos. Don't worry if the websites look ugly — we're just getting started.

HTML Anatomy

HTML is composed of [elements](#). These elements structure the webpage and define its content. Let's take a look at how they're written.

The diagram to the right displays an HTML paragraph element. As we can see, the paragraph element is made up of:

An *opening tag* (`<p>`)

The *content* ("Hello World!" text)

A *closing tag* (`</p>`)

A *tag* and the *content* between it is called an HTML element. There are many tags that we can use to organize and display text and other types of content, like images.



Let's quickly review each part of the element pictured:

HTML element (or simply, element) – a unit of content in an HTML document formed by HTML tags and the text or media it contains.

HTML Tag – the element name, surrounded by an opening (<) and closing (>) angle bracket.

Opening Tag – the first HTML tag used to start an HTML element. The tag type is surrounded by opening and closing angle brackets.

Content – The information (text or other elements) contained between the opening and closing tags of an HTML element.

Closing tag – the second HTML tag used to end an HTML element.

Closing tags have a forward slash (/) inside of them, directly after the left angle bracket.

The Body

One of the key HTML elements we use to build a webpage is the *body* element. Only content inside the opening and closing body tags can be displayed to the screen. Here's what opening and closing body tags look like:

```
<body>  
</body>
```

Once the file has a body, many different types of content – including text, images, and buttons – can be added to the body.

```
<body>  
  <p>What 's up, doc?</p>  
</body>
```

Instructions

1. Add a body to your web page using the `<body>` element.

2. Add the following code between your opening and closing body tags:

```
<p>"Life is very short and what we have to do must be  
done in the now." - Audre Lorde</p>
```

Note: While some browsers may render some content outside body tags as well to accommodate the occasional mistake in your HTML, not all browsers do this! The best way to ensure that all your HTML renders the same way in all browsers is to ensure that your elements remain within the opening and closing body tags.

```
<body><p>"Life is very short and what we have to do must be done  
in the now." - Audre Lorde</p> </body>
```

HTML Structure

HTML is organized as a collection of family tree relationships. As you saw in the last exercise, we placed `<p>` tags within `<body>` tags. When an element is contained inside another element, it is considered the *child* of that element. The child element is said to be *nested* inside the *parent* element.

```
<body>  
  <p>This paragraph is a child of the body</p>  
</body>
```

In the example above, the `<p>` element is nested inside the `<body>` element. The `<p>` element is considered a child of the `<body>` element, and the `<body>` element is considered the parent. You can also see that we've added two spaces of indentation (using the `space` bar) for better readability.

Since there can be multiple levels of nesting, this analogy can be extended to grandchildren, great-grandchildren, and beyond. The relationship between elements and their ancestor and descendant elements is known as *hierarchy*.

Let's consider a more complicated example that uses some new tags:

```
<body>

  <div>

    <h1>Sibling to p, but also grandchild of body</h1>

    <p>Sibling to h1, but also grandchild of body</p>

  </div>

</body>
```

In this example, the `<body>` element is the parent of the `<div>` element. Both the `<h1>` and `<p>` elements are children of the `<div>` element. Because the `<h1>` and `<p>` elements are at the same level, they are considered siblings and are both grandchildren of the `<body>` element.

Understanding HTML hierarchy is important because child elements can inherit behavior and styling from their parent element. You'll learn more about webpage hierarchy when you start digging into CSS.

Instructions

Checkpoint 1 Passed

1. Add the paragraph below as a child of the `div` element.

```
<p>This paragraph is a child of the div element and a  
grandchild of the body element</p>
```

```
<body>

  <h1>Hello World</h1>

  <p>This paragraph is a child of the body element</p>

  <div>

    <p>This paragraph is a child of the div element and a
grandchild of the body element</p>

  </div>

</body>
```

Headings

Headings in HTML are similar to headings in other types of media. For example, in newspapers, large headings are typically used to capture a reader's attention. Other times, headings are used to describe content, like the title of a movie or an educational article.

HTML follows a similar pattern. In HTML, there are six different *headings*, or *heading elements*. [Headings](#) can be used for a variety of purposes, like titling sections, articles, or other forms of content.

The following is the list of heading elements available in HTML. They are ordered from largest to smallest in size.

`<h1>` — used for main headings. All other smaller headings are used for subheadings.

`<h2>` `<h3>` `<h4>` `<h5>` `<h6>`

The following example code uses a headline intended to capture a reader's attention. It uses the largest heading available, the main heading element:

```
<h1>BREAKING NEWS</h1>
```

Instructions

1.Now that you know how to structure HTML elements, we'll spend the rest of the lesson building an informational website using some of the most common HTML elements. We've put some elements in to get you started, but you'll write the rest of the page on your own.Below the `<h3>` heading that says `Features`, add an `<h2>` heading that says `Habitat`.

2.Below the `Habitat` heading, add an `<h3>` heading that says `Countries with Large Brown Bear Populations`.

3.On the next line, add one more `<h3>` heading that says `Countries with Small Brown Bear Populations`.

4.Finally, on the next line add an `<h2>` heading that says `Media`.

```
<body>

<h1>The Brown Bear</h1>

<h2>About Brown Bears</h2>

<h3>Species</h3>

<h3>Features</h3>

<h2>Habitat</h2>

<h3>Countries with Large Brown Bear Populations</h3>

<h3>Countries with Small Brown Bear Populations</h3>
```

```
<h2>Media</h2>
```

```
</body>
```

Divs

One of the most popular elements in HTML is the `<div>` element. `<div>` is short for “division” or a container that divides the page into sections. These sections are very useful for grouping elements in your HTML together.

```
<body>
```

```
  <div>
```

```
    <h1>Why use divs?</h1>
```

```
    <p>Great for grouping elements!</p>
```

```
  </div>
```

```
</body>
```

`<div>`s don’t inherently have a visual representation, but they are very useful when we want to apply custom styles to our HTML elements. `<div>`s allow us to group HTML elements to apply the same styles for all HTML elements inside. We can also style the `<div>` element as a whole. You can see how this can be done in the Learn CSS course.

`<div>`s can contain any text or other HTML elements, such as links, images, or videos. Remember to always add two spaces of indentation when you nest elements inside of `<div>`s for better readability.

Instructions

1. Below the `<h1>` heading that says `The Brown Bear`, add an opening `<div>` tag. Place the closing `</div>` tag after the `<h3>` element that says `Features`. Remember to use your space bar to add two spaces of indentation when you nest elements.

2. Above the `<h2>` element that says `Habitat`, add an opening `<div>` tag. Close the `</div>` tag after the `<h3>` element that says `Countries with Small Brown Bear Populations`.

3. Above the `<h2>` element that says `Media`, add an opening `<div>` tag. Place the closing `</div>` tag right above the closing `</body>` tag.

Attributes

If we want to expand an element's tag, we can do so using an attribute. [Attributes](#) are content added to the opening tag of an element and can be used in several different ways, from providing information to changing styling. Attributes are made up of the following two parts:

The *name* of the attribute

The *value* of the attribute

One commonly used attribute is the `id`. We can use the `id` attribute to specify different content (such as `<div>`s) and is really helpful when you use an element more than once. `ids` have several different purposes in HTML, but for now, we'll focus on how they can help us identify content on our page.

When we add an `id` to a `<div>`, we place it in the opening tag:

```
<div id="intro">
  <h1>Introduction</h1>
</div>
```

Instructions

1. Add an `id` attribute with the value `"introduction"` to the `<div>` tag that's below the `The Brown Bear` `<h1>` heading.

2. Add an `id` attribute with the value `"habitat"` to the opening `<div>` tag that has the `Habitat` `<h2>` heading as a child.

3. Add an `id` attribute with the value `"media"` to the opening `<div>` tag that has the `Media` `<h2>` heading as a child.

Displaying Text

If you want to display text in HTML, you can use a *paragraph* or *span*:

[Paragraphs](#) (`<p>`) contain a block of plain text.

`<span>` contains short pieces of text or other HTML. They are used to separate small pieces of content that are on the same line as other content.

Take a look at each of these elements in action below:

```
<div>

  <h1>Technology</h1>

</div>

<div>

  <p><span>Self-driving cars</span> are anticipated to replace
up to 2 million jobs over the next two decades.</p>

</div>
```

In the example above, there are two different `<div>` elements. The second `<div>` contains a `<p>` with `<span>Self-driving cars</span>`. This `<span>` element separates “Self-driving cars” from the rest of the text in the paragraph.

It’s best to use a `<span>` element when you want to target a specific piece of content that is *inline*, or on the same line as other text. If you want to divide your content into *blocks*, it’s better to use a `<div>`.

Instructions

1. Below the `<h2>` element that says `About Brown Bears`, add `<p>` opening and closing tags, and inside of the tags put the following text: *The brown bear (Ursus arctos) is native to parts of northern Eurasia and North America. Its conservation status is currently Least Concern. There are many subspecies within the brown bear species, including the Atlas bear and the Himalayan brown bear.*

Remember to always add two spaces of indentation when you nest elements inside of `<div>`s for better readability.

2. Below the `<h3>` element that says `Features`, add a paragraph with the following text:

Brown bears are not always completely brown. Some can be reddish or yellowish. They have very large, curved claws and huge paws. Male brown bears are often 30% larger than female brown bears. They can range from 5 feet to 9 feet from head to toe.

3. Under the `<h3>` element that says `Countries with Small Brown Bear Populations`

Add a paragraph with the following text: *Some countries with smaller brown bear populations include Armenia, Belarus, Bulgaria, China, Finland, France, Greece, India, Japan, Nepal, Poland, Romania, Slovenia, Turkmenistan, and Uzbekistan.*

`<body>`



```
<h1>The Brown Bear</h1>
```

```
<div id="introduction">
```

```
<h2>About Brown Bears</h2>
```

```
<p>The brown bear (Ursus arctos) is native to parts of northern  
Eurasia and North America. Its conservation status is currently Least  
Concern. There are many subspecies within the brown bear species,  
including the Atlas bear and the Himalayan brown bear.</p>
```

```
<h3>Species</h3>
```

```
<h3>Features</h3>
```

```
<p>Brown bears are not always completely brown. Some can be reddish or  
yellowish. They have very large, curved claws and huge paws. Male brown  
bears are often 30% larger than female brown bears. They can range from 5  
feet to 9 feet from head to toe.</p>
```

```
</div>
```

```
<div id="habitat">
```

```
<h2>Habitat</h2>
```

```
<h3>Countries with Large Brown Bear Populations</h3>
```

```
<h3>Countries with Small Brown Bear Populations</h3>
```

```
<p>Some countries with smaller brown bear populations include Armenia,  
Belarus, Bulgaria, China, Finland, France, Greece, India, Japan, Nepal,  
Poland, Romania, Slovenia, Turkmenistan, and Uzbekistan.</p>  
  
</div>  
  
<div id= "media">  
  
<h2>Media</h2>  
  
</div>  
  
</body>
```

Styling Text

You can also style text using HTML tags. The `<em>` tag emphasizes text, while the `<strong>` tag highlights important text.

Later, when you begin to style websites, you will decide how you want browsers to display content within `<em>` and `<strong>` tags. Browsers, however, have built-in style sheets that will generally style these tags in the following ways:

The `<em>` tag will generally render as *italic* emphasis.

The `<strong>` will generally render as **bold** emphasis.

Take a look at each style in action:

```
<p><strong>The Nile River</strong> is the <em>longest</em> river  
in the world, measuring over 6,850 kilometers long  
(approximately 4,260 miles).</p>
```

In this example, the `<strong>` and `<em>` tags are used to emphasize the text to produce the following:

The Nile River is the *longest* river in the world, measuring over 6,850 kilometers long (approximately 4,260 miles).

As we can see, “The Nile River” is bolded and “longest” is in italics.

Instructions

1. In the first paragraph that starts “The brown bear...”, emphasize `Ursus arctos` using the `<em>` tag.

2. In the paragraph under `About Brown Bears`, make the words `Least Concern` strong using the `<strong>` tag.

```
<body>  
  
<h1>The Brown Bear</h1>  
  
<div id="introduction">  
  
<h2>About Brown Bears</h2>  
  
<p>The brown bear<em>(Ursus arctos)</em> is native to parts of  
northern Eurasia and North America. Its conservation status is currently  
<strong>Least Concern</strong>. There are many subspecies within the brown  
bear species, including the Atlas bear and the Himalayan brown bear.</p>  
  
<h3>Species</h3>  
  
<h3>Features</h3>
```

```
<p>Brown bears are not always completely brown. Some can be reddish or  
yellowish. They have very large, curved claws and huge paws. Male brown  
bears are often 30% larger than female brown bears. They can range from 5  
feet to 9 feet from head to toe.</p>  
  
</div>  
  
<div id="habitat">  
  
<h2>Habitat</h2>  
  
<h3>Countries with Large Brown Bear Populations</h3>  
  
<h3>Countries with Small Brown Bear Populations</h3>  
  
<p>Some countries with smaller brown bear populations include Armenia,  
Belarus, Bulgaria, China, Finland, France, Greece, India, Japan, Nepal,  
Poland, Romania, Slovenia, Turkmenistan, and Uzbekistan.</p>  
  
</div>  
  
<div id="media">  
  
<h2>Media</h2>  
  
</div>  
  
</body>
```

Line Breaks

The spacing between code in an HTML file doesn't affect the positioning of elements in the browser. If you are interested in modifying the spacing in the browser, you can use HTML's *line break* element: `<br>`.

The line break element is unique because it is only composed of a starting tag. You can use it anywhere within your HTML code and a line break will be shown in the browser.

```
<p>The Nile River is the longest river <br> in the world,  
measuring over 6,850 <br> kilometers long (approximately 4,260  
<br> miles).</p>
```

The code in the example above will result in an output that looks like the following:

The Nile River is the longest river in the world, measuring over 6,850 kilometers long (approximately 4,260 miles).

Instructions

1. Add two line breaks (
) after the sentence that ends with `Least Concern`.

```
<body>  
  
<h1>The Brown Bear</h1>  
  
<div id="introduction">  
  
<h2>About Brown Bears</h2>  
  
<p>The brown bear (<em>Ursus arctos</em>) is native to parts of  
northern Eurasia and North America. Its conservation status is currently  
<strong>Least Concern</strong>.<br><br> There are many subspecies within  
the brown bear species, including the Atlas bear and the Himalayan brown  
bear.</p>  
  
<h3>Species</h3>  
  
<h3>Features</h3>  
  
<p>Brown bears are not always completely brown. Some can be reddish or  
yellowish. They have very large, curved claws and huge paws. Male brown
```



```
bears are often 30% larger than female brown bears. They can range from 5
feet to 9 feet from head to toe.</p>

</div>

<div id="habitat">

  <h2>Habitat</h2>

  <h3>Countries with Large Brown Bear Populations</h3>

  <h3>Countries with Small Brown Bear Populations</h3>

  <p>Some countries with smaller brown bear populations include Armenia,
Belarus, Bulgaria, China, Finland, France, Greece, India, Japan, Nepal,
Poland, Romania, Slovenia, Turkmenistan, and Uzbekistan.</p>

</div>

<div id="media">

  <h2>Media</h2>

</div>

</body>
```

Unordered Lists

In addition to organizing text in paragraph form, you can also display content in an easy-to-read list.

In HTML, you can use an *unordered list* tag (`<ul>`) to create a list of items in no particular order. An unordered list outlines individual *list items* with a bullet point.

The `<ul>` [element](#) should not hold raw text and won't automatically format raw text into an unordered list of items. Individual list items must be added to the unordered list using the `<li>` tag. The `<li>` or list item tag is used to describe an item in a list.

```
<ul>

  <li>Limes</li>

  <li>Tortillas</li>

  <li>Chicken</li>

</ul>
```

In the example above, the list was created using the `<ul>` tag and all individual list items were added using `<li>` tags.

The output will look like this:

- Limes
- Tortillas
- Chicken

Instructions

1. Under the heading that says `Species`, create an unordered list. Do not add any list items to the list just yet.
2. Add the following list items to the unordered list: Arctos Collarus Horribilis Nelsoni (extinct)

```
<body>

  <h1>The Brown Bear</h1>

  <div id="introduction">
```

About Brown Bears

The brown bear (*Ursus arctos*) is native to parts of northern Eurasia and North America. Its conservation status is currently **Least Concern**. There are many subspecies within the brown bear species, including the Atlas bear and the Himalayan brown bear.

Species

-

- Arctos

- Collarus

- Horribilis

- Nelsoni (extinct)

Features

Brown bears are not always completely brown. Some can be reddish or yellowish. They have very large, curved claws and huge paws. Male brown bears are often 30% larger than female brown bears. They can range from 5 feet to 9 feet from head to toe.

Habitat

Countries with Large Brown Bear Populations

Countries with Small Brown Bear Populations

```
<p>Some countries with smaller brown bear populations include Armenia,
Belarus, Bulgaria, China, Finland, France, Greece, India, Japan, Nepal,
Poland, Romania, Slovenia, Turkmenistan, and Uzbekistan.</p>

</div>

<div id="media">

  <h2>Media</h2>

</div>

</body>
```

Ordered Lists

[*Ordered lists*](#) (`<ol>`) are like unordered lists, except that each list item is numbered. They are useful when you need to list different steps in a process or rank items from first to last.

You can create the ordered list with the `<ol>` tag and then add individual list items to the list using `<li>` tags.

```
<ol>

  <li>Preheat the oven to 350 degrees.</li>

  <li>Mix whole wheat flour, baking soda, and salt.</li>

  <li>Cream the butter and sugar in a separate bowl.</li>

  <li>Add eggs and vanilla extract to the bowl.</li>

</ol>
```

Copy to Clipboard

The output will look like this:

Preheat the oven to 350 degrees.

Mix whole wheat flour, baking soda, and salt.

Cream the butter and sugar in a separate bowl.

Add eggs and vanilla extract to the bowl.

Instructions

1. Under the heading that says `Countries with Large Brown Bear Populations`, add an ordered list. Do not add any list items to the list just yet.

2. Add the following list items to the ordered list: Russia United States Canada

```
<body>

<h1>The Brown Bear</h1>

<div id="introduction">

  <h2>About Brown Bears</h2>

  <p>The brown bear (<em>Ursus arctos</em>) is native to parts of
northern Eurasia and North America. Its conservation status is currently
<strong>Least Concern</strong>.<br /><br /> There are many subspecies
within the brown bear species, including the Atlas bear and the Himalayan
brown bear.</p>

  <h3>Species</h3>

  <ul>

    <li>Arctos</li>
```

- Collarus

- Horribilis

- Nelsoni (extinct)

-

<h3>Features</h3>

<p>Brown bears are not always completely brown. Some can be reddish or yellowish. They have very large, curved claws and huge paws. Male brown bears are often 30% larger than female brown bears. They can range from 5 feet to 9 feet from head to toe.</p>

</div>

<div id="habitat">

<h2>Habitat</h2>

<h3>Countries with Large Brown Bear Populations</h3>

-

- Russia

- United States

- Canada

-

<h3>Countries with Small Brown Bear Populations</h3>

<p>Some countries with smaller brown bear populations include Armenia, Belarus, Bulgaria, China, Finland, France, Greece, India, Japan, Nepal, Poland, Romania, Slovenia, Turkmenistan, and Uzbekistan.</p>

```
</div>

<div id="media">

  <h2>Media</h2>

</div>

</body>
```

Images

All of the elements you've learned about so far (headings, paragraphs, lists, and spans) share one thing in common: they're composed entirely of text! What if you want to add content to your web page that isn't composed of text, like [images](#)?

The `<img>` tag allows you to add an image to a web page. Most elements require both opening and closing tags, but the `<img>` tag is a *self-closing tag*. Note that the end of the `<img>` tag has a forward slash `/`. Self-closing tags may include or omit the final slash — both will render properly.

```

```

The `<img>` tag has a required *attribute* called `src`. The `src` attribute must be set to the image's *source*, or the location of the image. In this case, the value of `src` must be the *uniform resource locator* (URL) of the image. A URL is the web address or local address where a file is stored.

Instructions

1. Under the `Media` `<h2>` heading, add an image. Use the following URL as the source (`src`) for the

image: https://content.codecademy.com/courses/web-101/web101-image_brownbear.jpg

Image Alts

Part of being an exceptional web developer is making your site accessible to users of all backgrounds. In order to make the Web more inclusive, we need to consider what happens when assistive technologies such as screen readers come across image tags.

The `alt` attribute, which means alternative text, brings meaning to the images on our sites. The `alt` attribute can be added to the image tag just like the `src` attribute. The value of `alt` should be a description of the image.

```

```

The `alt` attribute also serves the following purposes: If an image fails to load on a web page, a user can mouse over the area originally intended for the image and read a brief description of the image. This is made possible by the description you provide in the `alt` attribute. Visually impaired users often browse the web with the aid of screen reading software. When you include the `alt` attribute, the screen reading software can read the image's description out loud to the visually impaired user. The `alt` attribute also plays a role in Search Engine Optimization (SEO), because search engines cannot "see" the images on websites as they crawl the internet. Having descriptive `alt` attributes can improve the ranking of your site. If the image on the web page is not one that conveys any meaningful information to a user (visually impaired or otherwise), the `alt` attribute should be left empty.

Instructions

1. Add the `alt` attribute to the image and include a description. Make sure the description accurately describes the image.

```
<body>  
  
<h1>The Brown Bear</h1>  
  
<div id="introduction">
```


About Brown Bears

The brown bear (*Ursus arctos*) is native to parts of northern Eurasia and North America. Its conservation status is currently **Least Concern**.
There are many subspecies within the brown bear species, including the Atlas bear and the Himalayan brown bear.

Species

-

- Arctos

- Collarus

- Horribilis

- Nelsoni (extinct)

Features

Brown bears are not always completely brown. Some can be reddish or yellowish. They have very large, curved claws and huge paws. Male brown bears are often 30% larger than female brown bears. They can range from 5 feet to 9 feet from head to toe.

Habitat

Countries with Large Brown Bear Populations

-

```
<li>Russia</li>
```

```
<li>United States</li>
```

```
<li>Canada</li>
```

```
</ol>
```

```
<h3>Countries with Small Brown Bear Populations</h3>
```

```
<p>Some countries with smaller brown bear populations include Armenia,  
Belarus, Bulgaria, China, Finland, France, Greece, India, Japan, Nepal,  
Poland, Romania, Slovenia, Turkmenistan, and Uzbekistan.</p>
```

```
</div>
```

```
<div id="media">
```

```
<h2>Media</h2>
```

```

```

```
</div>
```

```
</body>
```

Videos

In addition to images, HTML also supports displaying [videos](#). Like the `<img>` element, the `<video>` element requires a `src` attribute with a link to the video source. Unlike the `<img>` element however, the `<video>` element requires an opening and a closing tag.

```
<video src="myVideo.mp4" width="320" height="240" controls>
  Video not supported
</video>
```

In this example, the video source (`src`) is `"myVideo.mp4"`. The source can be a video file that is hosted alongside your webpage, or a URL that points to a video file hosted on another webpage.

After the `src` attribute, the `width` and `height` attributes are used to set the size of the video displayed in the browser. The `controls` attribute instructs the browser to include basic video controls such as pausing and playing.

The text, `Video not supported`, between the opening and closing video tags will only be displayed if the browser is unable to load the video.

Instructions

1. Under the image, create a `<video>` tag and add the following video url as the source:

```
https://content.codecademy.com/courses/freelance-1/unit-1/lesson-2/htmlcss1-vid_brown-bear.mp4
```

Be sure to create a closing tag as well with `</video>`.

2. Define the `width` of the video as `"320"` and the `height` as `"240"`. Make sure to also include the `controls` attribute.

3. In between the opening and closing `<video>` tags, add the phrase `Video not supported`, which will be displayed if the browser is unable to load your video.

```
<body>

  <h1>The Brown Bear</h1>

  <div id="introduction">

    <h2>About Brown Bears</h2>

    <p>The brown bear (Ursus arctos) is native to parts of
    northern Eurasia and North America. Its conservation status is currently
    <strong>Least Concern</strong>.<br /><br /> There are many subspecies
    within the brown bear species, including the Atlas bear and the Himalayan
    brown bear.</p>

    <h3>Species</h3>

    <ul>

      <li>Arctos</li>

      <li>Collarus</li>

      <li>Horribilis</li>

      <li>Nelsoni (extinct)</li>

    </ul>

    <h3>Features</h3>

    <p>Brown bears are not always completely brown. Some can be reddish or
    yellowish. They have very large, curved claws and huge paws. Male brown
    bears are often 30% larger than female brown bears. They can range from 5
    feet to 9 feet from head to toe.</p>
```

```
</div>
```

```
<div id="habitat">
```

```
<h2>Habitat</h2>
```

```
<h3>Countries with Large Brown Bear Populations</h3>
```

```
<ol>
```

```
<li>Russia</li>
```

```
<li>United States</li>
```

```
<li>Canada</li>
```

```
</ol>
```

```
<h3>Countries with Small Brown Bear Populations</h3>
```

```
<p>Some countries with smaller brown bear populations include Armenia, Belarus, Bulgaria, China, Finland, France, Greece, India, Japan, Nepal, Poland, Romania, Slovenia, Turkmenistan, and Uzbekistan.</p>
```

```
</div>
```

```
<div id="media">
```

```
<h2>Media</h2>
```

```

```

```
<video
```

```
src="https://content.codecademy.com/courses/freelance-1/unit-1/lesson-2/htmlcss1-vid_brown-bear.mp4" width="320" height="240" controls>
```

```
Video not supported

</video>

</div>

</body>
```

HTML Document Standards

Preparing for HTML

Now that we've learned about some of the most common HTML elements, it's time to learn how to set up an HTML file.

HTML files require certain elements to set up the document properly. We can let web browsers know that we are using HTML by starting our document with a *document type declaration*.

The declaration looks like this:

```
<!DOCTYPE html>
```

This declaration is an instruction, and it must be the first line of code in your HTML document. It tells the browser what type of document to expect, along with what version of HTML is being used in the document. For now, the browser will correctly assume that the `html` in `<!DOCTYPE html>` is referring to HTML5, as it is the current standard.

In the future, however, a new standard will override HTML5. To make sure your document is forever interpreted correctly, always include `<!DOCTYPE html>` at the very beginning of your HTML documents.

Lastly, HTML code is always saved in a file with an **.html** extension.

Instructions

1. Add the `<!DOCTYPE html>` declaration as the very first line of code at the top of the `index.html` file.

```
<!DOCTYPE html>
```

The `<html>` Tag

The `<!DOCTYPE html>` declaration provides the browser with two pieces of information (the type of document and the HTML version to expect), but it doesn't actually add any HTML structure or content.

To create HTML structure and content, we must add opening and closing `<html>` tags after declaring `<!DOCTYPE html>`:

```
<!DOCTYPE html>
<html>

</html>
```

Anything between the opening `<html>` and closing `</html>` tags will be interpreted as HTML code. Without these tags, it's possible that browsers could incorrectly interpret your HTML code.

Instructions

1. Add the opening and closing `<html>` tags below the `<!DOCTYPE html>` tag.

```
<!DOCTYPE html>

<html></html>
```

The Head

So far you've done two things to set up the file properly:

Declared to the browser that your code is HTML with `<!DOCTYPE html>`

Added the HTML element (`<html>`) that will contain the rest of your code.

We have added these elements to the Brown Bears page you previously created. Now, let's also give the browser some information about the page itself. We can do this by adding a `<head>` element.

Remember the `<body>` tag? The `<head>` element is part of this HTML metaphor. It goes above our `<body>` element.

The `<head>` element contains the *metadata* for a web page. Metadata is information about the page that isn't displayed directly on the web page. Unlike the information inside of the `<body>` tag, the metadata in the head is information about the page itself. You'll see an example of this in the next exercise.

The opening and closing head tags typically appear as the first item after your first HTML tag:

```
<head>  
  
</head>
```

Instructions

1. Place the `<head>` element below the opening `<html>` tag.

```
<!DOCTYPE html>  
  
<html>  
  
  <head>  
  
  </head>
```



```
<body>

  <h1>The Brown Bear</h1>

  <div id="introduction">

    <h2>About Brown Bears</h2>

    <p>The brown bear (<em>Ursus arctos</em>) is native to parts of
    northern Eurasia and North America. Its conservation status is currently
    <strong>Least Concern</strong>.<br /><br /> There are many subspecies
    within the brown bear species, including the

      Atlas bear and the Himalayan brown bear.</p>

    <h3>Species</h3>

    <ul>

      <li>Arctos</li>

      <li>Collarus</li>

      <li>Horribilis</li>

      <li>Nelsoni (extinct)</li>

    </ul>

    <h3>Features</h3>

    <p>Brown bears are not always completely brown. Some can be reddish
    or yellowish. They have very large, curved claws and huge paws. Male brown
    bears are often 30% larger than female brown bears. They can range from 5
    feet to 9 feet from head to toe.</p>

  </div>
```

```
<div id="habitat">

  <h2>Habitat</h2>

  <h3>Countries with Large Brown Bear Populations</h3>

  <ol>

    <li>Russia</li>

    <li>United States</li>

    <li>Canada</li>

  </ol>

  <h3>Countries with Small Brown Bear Populations</h3>

  <p>Some countries with smaller brown bear populations include Armenia, Belarus, Bulgaria, China, Finland, France, Greece, India, Japan, Nepal, Poland, Romania, Slovenia, Turkmenistan, and Uzbekistan.</p>

</div>

<div id="media">

  <h2>Media</h2>

  

  <video
src="https://content.codecademy.com/courses/freelance-1/unit-1/lesson-2/htmlcss1-vid_brown-bear.mp4" width="320" height="240" controls>Video not supported</video>

</div>
```

```
</body>

</html>
```

Page Titles

What kind of metadata about the web page can the `<head>` element contain?

If you navigate to the Codecademy catalog and look at the top of your browser, you'll notice the words `Catalog Home | Codecademy`, which is the *title* of the web page.

A browser's tab displays the title specified in the `<title>` tag. The `<title>` tag is always inside of the `<head>`.

```
<!DOCTYPE html>

<html>

  <head>

    <title>My Coding Journal</title>

  </head>

</html>
```

If we were to open a file containing the HTML code in the example above, the browser would display the words `My Coding Journal` in the title bar (or in the tab's title).

Instructions

1. Add "Brown Bears" as the title of your web page by using the `<title>` element. Unfortunately, you won't be able to see the title of your page in the smaller browser to the right. We'll show you what it would look like in the next exercise.

```
<!DOCTYPE html>

<html>

  <head>

    <title>Brown Bears </title>

  </head>

  <body>

    <h1>The Brown Bear</h1>

    <div id="introduction">

      <h2>About Brown Bears</h2>

      <p>The brown bear (<em>Ursus arctos</em>) is native to parts of northern Eurasia and North America. Its conservation status is currently <strong>Least Concern</strong>.<br /><br /> There are many subspecies within the brown bear species, including the Atlas bear and the Himalayan brown bear.</p>

      <h3>Species</h3>

      <ul>

        <li>Arctos</li>

        <li>Collarus</li>

        <li>Horribilis</li>
```

```
<li>Nelsoni (extinct)</li>
```

```
</ul>
```

```
<h3>Features</h3>
```

```
<p>Brown bears are not always completely brown. Some can be reddish or yellowish. They have very large, curved claws and huge paws. Male brown bears are often 30% larger than female brown bears. They can range from 5 feet to 9 feet from head to toe.</p>
```

```
</div>
```

```
<div id="habitat">
```

```
<h2>Habitat</h2>
```

```
<h3>Countries with Large Brown Bear Populations</h3>
```

```
<ol>
```

```
<li>Russia</li>
```

```
<li>United States</li>
```

```
<li>Canada</li>
```

```
</ol>
```

```
<h3>Countries with Small Brown Bear Populations</h3>
```

```
<p>Some countries with smaller brown bear populations include Armenia, Belarus, Bulgaria, China, Finland, France, Greece, India, Japan, Nepal, Poland, Romania, Slovenia, Turkmenistan, and Uzbekistan.</p>
```

```
</div>
```

```
<div id="media">
```

```

    <h2>Media</h2>

    

    <video
src="https://content.codecademy.com/courses/freelance-1/unit-1/lesson-2/ht
mlcss1-vid_brown-bear.mp4" width="320" height="240" controls>Video not
supported</video>

  </div>

</body>

</html>

```

Where Does the Title Appear?

Good work! Unfortunately, the browser panel used by the Codecademy environment does not have a title bar, so the “Brown Bear” title you wrote in the previous exercise will not be displayed. Outside of the Codecademy environment, however, your title would appear as depicted in the diagram to the right.

So far, we have learned about:

- `<!DOCTYPE html>`, the declaration specifying the version of HTML for the browser

- The `<html>` tags that enclose all of your HTML code

- The `<head>` tag that contains the metadata of a webpage, such as its `<title>`

Next, you will learn about new types of elements that go inside the body.

Linking to Other Web Pages

One of the powerful aspects of HTML (and the Internet), is the ability to *link* to other web pages.

You can add links to a web page by adding an *anchor* element `<a>` and including the text of the link in between the opening and closing tags.

```
<a>This Is A Link To Wikipedia</a>
```

Copy to Clipboard

Wait a minute! Technically, the link in the example above is incomplete. How exactly is the link above supposed to work if there is no URL that will lead users to the actual Wikipedia page?

The anchor element in the example above is incomplete without the `href` attribute. This attribute stands for *hypertext reference* and is used to link to a *path*, or the address to where a file is located (whether it is on your computer or another location). The paths provided to the `href` attribute are often URLs.

```
<a href="https://www.wikipedia.org/">This Is A Link To  
Wikipedia</a>
```

Copy to Clipboard

In the example above, the `href` attribute has been set to the value of the URL `https://www.wikipedia.org/`. The example now shows the correct use of an anchor element.

When reading technical documentation, you may come across the term *hyperlink*. Not to worry, this is simply the technical term for link. These terms are often used interchangeably.

Instructions

1. After the first paragraph, right above the heading that says `Species`, add a link that says: `Learn More`. Don't add the URL just yet.

2. Add the following URL to the link:

```
https://en.wikipedia.org/wiki/Brown_bear
```

```
<!DOCTYPE html>

<html>

  <head>

    <title>Brown Bears</title>

  </head>

  <body>

    <h1>The Brown Bear</h1>

    <div id="introduction">

      <h2>About Brown Bears</h2>

      <p>The brown bear (<em>Ursus arctos</em>) is native to parts of
northern Eurasia and North America. Its conservation status is currently
<strong>Least Concern</strong>.<br /><br /> There are many subspecies
within the brown bear species, including the

      Atlas bear and the Himalayan brown bear.</p>

      <a href="https://en.wikipedia.org/wiki/Brown_bear">Learn More</a>

    <h3>Species</h3>
```



```
<ul>
```

```
<li>Arctos</li>
```

```
<li>Collarus</li>
```

```
<li>Horribilis</li>
```

```
<li>Nelsoni (extinct)</li>
```

```
</ul>
```

```
<h3>Features</h3>
```

```
<p>Brown bears are not always completely brown. Some can be reddish  
or yellowish. They have very large, curved claws and huge paws. Male brown  
bears are often 30% larger than female brown bears. They can range from 5  
feet to 9 feet from head to toe.</p>
```

```
</div>
```

```
<div id="habitat">
```

```
<h2>Habitat</h2>
```

```
<h3>Countries with Large Brown Bear Populations</h3>
```

```
<ol>
```

```
<li>Russia</li>
```

```
<li>United States</li>
```

```
<li>Canada</li>
```

```
</ol>
```

```

<h3>Countries with Small Brown Bear Populations</h3>

<p>Some countries with smaller brown bear populations include
Armenia, Belarus, Bulgaria, China, Finland, France, Greece, India, Japan,
Nepal, Poland, Romania, Slovenia, Turkmenistan, and Uzbekistan.</p>

</div>

<div id="media">

  <h2>Media</h2>

  

  <video
src="https://content.codecademy.com/courses/freelance-1/unit-1/lesson-2/ht
mlcss1-vid_brown-bear.mp4" width="320" height="240" controls>Video not
supported</video>

</div>

</body>

</html>

```

Opening Links in a New Window

Have you ever clicked on a link and observed the resulting web page open in a new browser window? If so, you can thank the `<a>` element's `target` attribute.

The `target` attribute specifies how a link should open.

It's possible that one or more links on your web page link to an entirely different website. In that case, you may want users to read the linked website, but hope that they return to your web page. This is exactly when the `target` attribute is useful!

For a link to open in a new window, the `target` attribute requires a value of `_blank`. The `target` attribute can be added directly to the opening tag of the anchor element, just like the `href` attribute.

```
<a href="https://en.wikipedia.org/wiki/Brown_bear"
target="_blank">The Brown Bear</a>
```

In the example above, setting the `target` attribute to `"_blank"` instructs the browser to open the relevant Wikipedia page in a new window.

In this exercise, we've used the terminology "open in a new window." It's likely that you are using a modern browser that opens up websites in new *tabs*, rather than new windows. Before the advent of browsers with tabs, additional browser windows had to be opened to view more websites. The `target="_blank"` attribute, when used in modern browsers, will open new websites in a new tab.

Instructions

1. In the link that reads `Learn More`, add the `target` attribute and set it equal to `_blank`.

```
<!DOCTYPE html>

<html>

  <head>

    <title>Brown Bears</title>

  </head>

  <body>
```

<h1>The Brown Bear</h1>

<div id="introduction">

<h2>About Brown Bears</h2>

<p>The brown bear (Ursus arctos) is native to parts of northern Eurasia and North America. Its conservation status is currently Least Concern.

 There are many subspecies within the brown bear species, including the

Atlas bear and the Himalayan brown bear.</p>

Learn More

<h3>Species</h3>

Arctos

Collarus

Horribilis

Nelsoni (extinct)

<h3>Features</h3>

<p>Brown bears are not always completely brown. Some can be reddish or yellowish. They have very large, curved claws and huge paws. Male brown bears are often 30% larger than female brown bears. They can range from 5 feet to 9 feet from head to toe.</p>

</div>

```
<div id="habitat">

  <h2>Habitat</h2>

  <h3>Countries with Large Brown Bear Populations</h3>

  <ol>

    <li>Russia</li>

    <li>United States</li>

    <li>Canada</li>

  </ol>

  <h3>Countries with Small Brown Bear Populations</h3>

  <p>Some countries with smaller brown bear populations include Armenia, Belarus, Bulgaria, China, Finland, France, Greece, India, Japan, Nepal, Poland, Romania, Slovenia, Turkmenistan, and Uzbekistan.</p>

</div>

<div id="media">

  <h2>Media</h2>

  

  <video
src="https://content.codecademy.com/courses/freelance-1/unit-1/lesson-2/htmlcss1-vid_brown-bear.mp4" width="320" height="240" controls>Video not supported</video>

</div>
```

```
</body>

</html>
```

Linking to Relative Page

Thus far you have learned how to link to external web pages. Many sites also link to internal web pages like Home, About, and Contact.

Before we learn how to link between internal pages, let's establish where our files are stored. When making multi-page static websites, web developers often store HTML files in the *root directory*, or a main folder where all the files for the project are stored. As the size of the projects you create grows, you may use additional folders within the main project folder to organize your code.

```
project-folder/

|— about.html

|— contact.html

|— index.html
```

The example above shows three different files — **about.html**, **contact.html**, and **index.html** in one folder.

HTML files are often stored in the same folder, as shown in the example above. If the browser is currently displaying **index.html**, it also knows that **about.html** and **contact.html** are in the same folder. Because the files are stored in the same folder, we can link web pages together using a *relative path*.

```
<a href="./contact.html">Contact</a>
```

Copy to Clipboard

In this example, the `<a>` tag is used with a relative path to link from the current HTML file to the `contact.html` file in the same folder. On the web page, `Contact` will appear as a link.

A relative path is a filename that shows the path to a *local file* (a file on the same website, such as `./index.html`) versus an absolute path (a full URL, like `https://www.codecademy.com/learn/learn-html` which is stored in a different folder). The `./` in `./index.html` tells the browser to look for the file in the current folder.

Instructions

1. Directly below the opening `<body>` tag within **index.html**, add an anchor tag that links to **index.html** with a relative path. The link should say `Brown Bear`.
2. Under the link to **index.html**, add an anchor tag that links to **aboutme.html** using a relative path. The link should say `About Me`. Notice that there is also another file in the code editor, **aboutme.html**. Click the link to make sure it works!

```
<!DOCTYPE html>

<html>

<head>

  <title>Brown Bears</title>

</head>

<body>

  <a href="./index.html">Brown Bear</a>

  <a href="./aboutme.html">About Me</a>
```

<h1>The Brown Bear</h1>

<div id="introduction">

<h2>About Brown Bears</h2>

<p>The brown bear (Ursus arctos) is native to parts of northern Eurasia and North America. Its conservation status is currently Least Concern.

 There are many subspecies within the brown bear species, including the

Atlas bear and the Himalayan brown bear.</p>

Learn More

<h3>Species</h3>

Arctos

Collarus

Horribilis

Nelsoni (extinct)

<h3>Features</h3>

<p>Brown bears are not always completely brown. Some can be reddish or yellowish. They have very large, curved claws and huge paws. Male brown bears are often 30% larger than female brown bears. They can range from 5 feet to 9 feet from head to toe.</p>

</div>


```
<div id="habitat">

  <h2>Habitat</h2>

  <h3>Countries with Large Brown Bear Populations</h3>

  <ol>

    <li>Russia</li>

    <li>United States</li>

    <li>Canada</li>

  </ol>

  <h3>Countries with Small Brown Bear Populations</h3>

  <p>Some countries with smaller brown bear populations include Armenia, Belarus, Bulgaria, China, Finland, France, Greece, India, Japan, Nepal, Poland, Romania, Slovenia, Turkmenistan, and Uzbekistan.</p>

</div>

<div id="media">

  <h2>Media</h2>

  

  <video
src="https://content.codecademy.com/courses/freelance-1/unit-1/lesson-2/htmlcss1-vid_brown-bear.mp4" height="240" width="320" controls>Video not supported</video>

</div>
```

```
</body>

</html>
```

Linking At Will

You've probably visited websites where not all links were made up of text. Maybe the links you clicked on were images or some other form of content.

So far, we've added links that were made up of only text, like the following:

```
<a href="https://en.wikipedia.org/wiki/Opuntia"
target="_blank">Prickly Pear</a>
```

Text-only links, however, would significantly decrease your flexibility as a web developer!

Thankfully, HTML allows you to turn nearly any element into a link by wrapping that element with an anchor element. With this technique, it's possible to turn images into links by simply wrapping the `<img>` element with an `<a>` element.

```
<a href="https://en.wikipedia.org/wiki/Opuntia"
target="_blank"></a>
```

In the example above, an image of a prickly pear has been turned into a link by wrapping the outside of the `<img>` element with an `<a>` element.

Instructions

1. In `index.html`, transform the image on your page into a link by wrapping the image element with an anchor element. Set the `target` attribute so that the link opens in a new window. Your image element should be a nested tag inside your anchor element. Use the same URL as before:

https://en.wikipedia.org/wiki/Brown_bear

```
<!DOCTYPE html>

<html>

<head>

  <title>Brown Bears</title>

</head>

<body>

  <a href="./index.html">Brown Bear</a>

  <a href="./aboutme.html">About Me</a>

  <h1>The Brown Bear</h1>

  <div id="introduction">

    <h2>About Brown Bears</h2>

    <p>The brown bear (<em>Ursus arctos</em>) is native to parts of
northern Eurasia and North America. Its conservation status is currently
<strong>Least Concern</strong>.<br /><br /> There are many subspecies
within the brown bear species, including the

    Atlas bear and the Himalayan brown bear.</p>

    <a href="https://en.wikipedia.org/wiki/Brown_bear"
target="_blank">Learn More</a>
```

Species</h3>

Arctos

Collarus

Horribilis

Nelsoni (extinct)

Features</h3>

<p>Brown bears are not always completely brown. Some can be reddish or yellowish. They have very large, curved claws and huge paws. Male brown bears are often 30% larger than female brown bears. They can range from 5 feet to 9 feet from head to toe.</p>

</div>

<div id="habitat">

Habitat</h3>

Countries with Large Brown Bear Populations</h3>

Russia

United States

Canada


```

<h3>Countries with Small Brown Bear Populations</h3>

<p>Some countries with smaller brown bear populations include Armenia,
Belarus, Bulgaria, China, Finland, France, Greece, India, Japan, Nepal,
Poland, Romania, Slovenia, Turkmenistan, and Uzbekistan.</p>

</div>

<div id="media">

  <h2>Media</h2>

  <a href="https://en.wikipedia.org/wiki/Brown_bear"
target="_blank"></a>

  <video
src="https://content.codecademy.com/courses/freelance-1/unit-1/lesson-2/ht
mlcss1-vid_brown-bear.mp4" height="240" width="320" controls>Video not
supported</video>

</div>

</body>

</html>

```

Linking to Same Page

At this point, we have all the content we want on our page. Since we have so much content, it doesn't all fit on the screen. How do we make it easier for a user to jump to different portions of our page?

When users visit our site, we want them to be able to click a link and have the page automatically scroll to a specific section.

In order to link to a target on the same page, we must give the target an `id` attribute and a unique value like this:

```
<p id="top">This is the top of the page!</p>

<h1 id="bottom">This is the bottom! </h1>
```

In this example, the `<p>` element is assigned an `id` of “top” and the `<h1>` element is assigned “bottom.” An `id` can be added to most elements on a webpage.

An `id` should be descriptive to make it easier to remember the purpose of a link. The target link is a string containing the `#` character and the target element’s `id`.

```
<ol>

<li><a href="#top">Top</a></li>

<li><a href="#bottom">Bottom</a></li>

</ol>
```

In the example above, the links to `<p id="top">` and `<h1 id="bottom">` are embedded in an ordered list. These links appear in the browser as a numbered list of links. An `id` is especially helpful for organizing content belonging to a `div`!

Instructions

1. Under the `<h1>` element that says `The Brown Bear`, create an unordered list with the following three list items: Introduction Habitat Media

2. Wrap the text of each list item in the unordered list in an `anchor` element. Each anchor tag should link to the corresponding `<div>` on the page (The `<a>` element that contains the text “Introduction” links to `#introduction`).

```
<!DOCTYPE html>

<html>

<head>

  <title>Brown Bears</title>

</head>

<body>

  <a href="./index.html">Brown Bear</a>

  <a href="./aboutme.html">About Me</a>

  <h1>The Brown Bear</h1>

  <ul>

    <li><a href="#introduction">Introduction</a></li>

    <li><a href="#habitat">Habitat</a></li>

    <li><a href="#media">Media</a></li>

  </ul>

  <div id="introduction">

    <h2>About Brown Bears</h2>

    <p>The brown bear (<em>Ursus arctos</em>) is native to parts of northern Eurasia and North America. Its conservation status is currently <strong>Least Concern</strong>.<br /><br /> There are many subspecies within the brown bear species, including the

    Atlas bear and the Himalayan brown bear.</p>
```

```
<a href="https://en.wikipedia.org/wiki/Brown_bear"
target="_blank">Learn More</a>

<h3>Species</h3>

<ul>

  <li>Arctos</li>

  <li>Collarus</li>

  <li>Horribilis</li>

  <li>Nelsoni (extinct)</li>

</ul>

<h3>Features</h3>

<p>Brown bears are not always completely brown. Some can be reddish or
yellowish. They have very large, curved claws and huge paws. Male brown
bears are often 30% larger than female brown bears. They can range from 5
feet to 9 feet from head to toe.</p>

</div>

<div id="habitat">

  <h2>Habitat</h2>

  <h3>Countries with Large Brown Bear Populations</h3>

  <ol>

    <li>Russia</li>

    <li>United States</li>

    <li>Canada</li>
```



```

</ol>

<h3>Countries with Small Brown Bear Populations</h3>

<p>Some countries with smaller brown bear populations include Armenia,
Belarus, Bulgaria, China, Finland, France, Greece, India, Japan, Nepal,
Poland, Romania, Slovenia, Turkmenistan, and Uzbekistan.</p>

</div>

<div id="media">

  <h2>Media</h2>

  <a href="https://en.wikipedia.org/wiki/Brown_bear"
target="_blank"></a>

  <video
src="https://content.codecademy.com/courses/freelance-1/unit-1/lesson-2/ht
mlcss1-vid_brown-bear.mp4" height="240" width="320" controls>Video not
supported</video>

</div>

</body>

</html>

```

Whitespace

The rest of this lesson will focus on some tools developers use to make code easier to interpret.

As the code in an HTML file grows, it becomes increasingly difficult to keep track of how elements are related. Programmers use two tools to visualize the relationship between elements: *whitespace* and *indentation*.

Both tools take advantage of the fact that the position of elements in a browser is independent of the amount of whitespace or indentation in the **index.html** file.

For example, if you wanted to increase the space between two paragraphs on your web page, you would *not* be able to accomplish this by adding space between the paragraph elements in the **index.html** file. The browser ignores *whitespace* in HTML files when it renders a web page, so it can be used as a tool to make code easier to read and follow.

What makes the example below difficult to read?

```
<body><p>Paragraph 1</p><p>Paragraph 2</p></body>
```

You have to read the entire line to know what elements are present. Compare the example above to this:

```
<body>

    <p>Paragraph 1</p>

    <p>Paragraph 2</p>

</body>
```

This example is easier to read, because each element is on its own line. While the first example required you to read the entire line of code to identify the elements, this example makes it easy to identify the body tag and two paragraphs.

A browser renders both examples the same way:

```
Paragraph 1
```

Paragraph 2

In the next exercise you will learn how to use indentation to help visualize nested elements.

Instructions

1. Use whitespace to make the code more readable by putting each element on its own line.

```
<!DOCTYPE html>

<html>

<body>

    <h1>Whitespace</h1>

    <p> Whitespace and indentation make html documents easier to read</p>

</body>

</html>
```

Indentation

The second tool web developers use to make the structure of code easier to read is *indentation*. The spaces are inserted using the `space` and `tab` bars on your keyboard.

The [World Wide Web Consortium](https://www.w3.org/), or W3C, is responsible for maintaining the style standards of HTML. At the time of writing, the W3C recommends 2 spaces of indentation when writing HTML code. Although your code will work without exactly two spaces, this standard is followed by the majority of professional web developers. Indentation is used to easily visualize which elements are nested within other elements.

```
<body>

  <p>Paragraph 1</p>

  <div>

    <p>Paragraph 2</p>

  </div>

</body>
```

In the example above, `Paragraph 1` and the `<div>` tag are nested inside of the `<body>` tag, so they are indented two spaces. The `Paragraph 2` element is nested inside of the `<div>` tag, so it is indented an additional two spaces.

Instructions

1. Indent the code in **index.html** to match the W3C standards.

```
<body>

  <h1>Whitespace</h1>

  <div>

    <p>Whitespace and indentation make html documents easier to read.</p>

  </div>

</body>
```

Comments

HTML files also allow you to add [comments](#) to your code.

Comments begin with `<!--` and end with `-->`. Any characters in between will be ignored by your browser.

```
<!-- This is a comment that the browser will not display. -->
```

Including comments in your code is helpful for many reasons:

They help you (and others) understand your code if you decide to come back and review it at a much later date.

They allow you to experiment with new code, without having to delete old code.

```
<!-- Favorite Films Section -->  
  
<p>The following is a list of my favorite films:</p>
```

In this example, the comment is used to denote that the following text makes up a particular section of the page.

```
<!-- <p> Test Code </p> -->
```

In the example above, a valid HTML element (a paragraph element) has been “commented out.” This practice is useful when there is code you want to experiment with, or return to, in the future.

Instructions

1. Add a comment to **index.html** file.

```
<!DOCTYPE html>  
  
<html>  
  
  <head>  
  
    <!-- This document is about favourite Quotes. -->  
  
    <title>Favorite Quotes</title>  
  
  </head>
```

```
<body>

  <p>Feet, what do I need you for when I have wings to fly?</p>

  <p>- Frida Kahlo</p>

</body>

</html>
```

HTML Tables

Introduction to Tables

There are many websites on the Internet that display information like stock prices, sports scores, invoice data, and more. This data is tabular in nature, meaning that a table is often the best way of presenting the data.

In this part of the course, we'll learn how to use the HTML `<table>` element to present information in a two-dimensional table to the users.

Let's get started!

Instructions

Take a look at the code in the body of **index.html**. What new code do you see? Explore with the code in this file and try modifying something at random. What happened?

Create a Table

Before displaying data, we must first create the table that will contain the data by using the `<table>` element.

```
<table>

</table>
```

The `<table>` element will contain all of the tabular data we plan on displaying.

Instructions

1. In `index.html`, create a table just above the closing `body` tag.

Table Rows

In many programs that use tables, the table is already predefined for you, meaning that it contains the [rows, columns, and cells that will hold data](#). In HTML, all of these components must be created.

The first step in entering data into the table is to add rows using the *table row* element:

`<tr>`.

```
<table>

  <tr>

  </tr>

  <tr>

  </tr>

</table>
```

In the example above, two rows have been added to the table.

Instructions

1. Add two rows to the table in `index.html`.

Table Data

Rows aren't sufficient to add data to a table. Each cell element must also be defined. In HTML, you can add data using the *table data* element: `<td>`.

```
<table>
```

```
<tr>

  <td>73</td>

  <td>81</td>

</tr>

</table>
```

In the example above, two data points (73 and 81) were entered in the one row that exists. By adding two data points, we created two cells of data.

If the table were displayed in the browser, it would show a table with one row and two columns.

Instructions

1. In the second row, add three cells of data. The cells should contain the following data, in order: Adam's Greenworks,14,Package Items

Table Headings

Table data doesn't make much sense without titles to describe what the data represents.

To add titles to rows and columns, you can use the *table heading* element: `<th>`.

The table heading element is used just like a table data element, except with a relevant title. Just like table data, a table heading must be placed within a table row.

```
<table>

  <tr>

    <th></th>

    <th scope="col">Saturday</th>
```



```

    <th scope="col">Sunday</th>

</tr>

<tr>

    <th scope="row">Temperature</th>

    <td>73</td>

    <td>81</td>

</tr>
</table>

```

Copy to Clipboard

What happened in the code above?

First, a new row was added to hold the three headings: a blank heading, a `Saturday` heading, and a `Sunday` heading. The blank heading creates the extra table cell necessary to align the table headings correctly over the data they correspond to.

In the second row, one table heading was added as a row title: `Temperature`.

When rendered, this code will appear similar to the image below:

	Saturday	Sunday
Temperature	73	81

Note, also, the use of the `scope` attribute, which can take one of two values:

`row` - this value makes it clear that the heading is for a row.

`col` - this value makes it clear that the heading is for a column.

HTML code for tables may look a little strange at first, but analyzing it piece by piece helps make the code more understandable.

Instructions

1. In the first row, add three table headings. The headings should contain the following data, in order: Company Name, Number of Items to Ship, Next Action. These headings will add meaning to the rest of the data in the table.

2. Now add a `scope` attribute to each of these new headings.

Table Borders

So far, the tables you've created have been a little difficult to read because they have no borders.

In older versions of HTML, a border could be added to a table using the `border` attribute and setting it equal to an integer. This integer would represent the thickness of the border.

```
<table border="1">

  <tr>

    <td>73</td>

    <td>81</td>

  </tr>

</table>
```

The code in the example above is [deprecated](#), so please don't use it. It's meant to illustrate older conventions you may come across when reading other developers' code.

The browser will likely still interpret your code correctly if you use the `border` attribute, but that doesn't mean the attribute should be used.

We use CSS to add style to HTML documents, because it helps us to separate the structure of a page from how it looks. You can learn more about CSS in our CSS courses.

You can achieve the same table border effect using CSS.

```
table, td {  
  
    border: 1px solid black;  
  
}
```

The code in the example above uses CSS instead of HTML to show table borders.

Instructions

1. We're going to need some more data in the table. Add the following data to the table. Make sure to place it after the second table row.

```
<tr>  
    <td>Davie's Burgers</td>  
    <td>2</td>  
    <td>Send Invoice</td>  
</tr>  
<tr>  
    <td>Baker's Bike Shop</td>  
    <td>3</td>  
    <td>Send Invoice</td>  
</tr>  
<tr>  
    <td>Miss Sally's Southern</td>
```

```
<td>4</td>
<td>Ship</td>
</tr>
<tr>
<td>Summit Resort Rentals</td>
<td>4</td>
<td>Ship</td>
</tr>
<tr>
<td>Strike Fitness</td>
<td>1</td>
<td>Enter Order</td>
</tr>
```

Table Borders

So far, the tables you've created have been a little difficult to read because they have no borders.

In older versions of HTML, a border could be added to a table using the `border` attribute and setting it equal to an integer. This integer would represent the thickness of the border.

```
<table border="1">

  <tr>

    <td>73</td>

    <td>81</td>

  </tr>

</table>
```

The code in the example above is [deprecated](#), so please don't use it. It's meant to illustrate older conventions you may come across when reading other developers' code.

The browser will likely still interpret your code correctly if you use the `border` attribute, but that doesn't mean the attribute should be used.

We use CSS to add style to HTML documents, because it helps us to separate the structure of a page from how it looks. You can learn more about CSS in our CSS courses.

You can achieve the same table border effect using CSS.

```
table, td {  
  
    border: 1px solid black;  
  
}
```

The code in the example above uses CSS instead of HTML to show table borders.

Instructions

1. We're going to need some more data in the table. Add the following data to the table. Make sure to place it after the second table row.

```
<tr>  
    <td>Davie's Burgers</td>  
    <td>2</td>  
    <td>Send Invoice</td>  
</tr>  
<tr>  
    <td>Baker's Bike Shop</td>  
    <td>3</td>
```

```

        <td>Send Invoice</td>
    </tr>
    <tr>
        <td>Miss Sally's Southern</td>
        <td>4</td>
        <td>Ship</td>
    </tr>
    <tr>
        <td>Summit Resort Rentals</td>
        <td>4</td>
        <td>Ship</td>
    </tr>
    <tr>
        <td>Strike Fitness</td>
        <td>1</td>
        <td>Enter Order</td>
    </tr>

```

Spanning Columns

What if the table contains data that spans multiple columns?

For example, a personal calendar could have events that span across multiple hours, or even multiple days.

Data can span columns using the `colspan` attribute. The attribute accepts an integer (greater than or equal to 1) to denote the number of columns it spans across.

```

<table>
  <tr>
    <th>Monday</th>
    <th>Tuesday</th>
    <th>Wednesday</th>
  </tr>
  <tr>

```

```
<td colspan="2">Out of Town</td>
<td>Back in Town</td>
</tr>
</table>
```

In the example above, the data `Out of Town` spans the `Monday` and `Tuesday` table headings using the value `2` (two columns). The data `Back in Town` appears only under the `Wednesday` heading.

Instructions

1. In `index.html`, span a `<td>` element across two columns.

Table Body

Over time, a table can grow to contain a lot of data and become very long. When this happens, the table can be sectioned off so that it is easier to manage.

Long tables can be sectioned off using the *table body* element: `<tbody>`.

The `<tbody>` element should contain all of the table's data, excluding the table headings (more on this in a later exercise).

```
<table>
  <tbody>
    <tr>
      <th></th>
      <th>Saturday</th>
      <th>Sunday</th>
    </tr>
    <tr>
      <th>Morning</th>
      <td rowspan="2">Work</td>
      <td rowspan="3">Relax</td>
    </tr>
    <tr>
      <th>Afternoon</th>
```

```
<tr>
  <th>Evening</th>
  <td>Dinner</td>
</tr>
</tbody>
</table>
```

In the example above, all of the table data is contained within a table body element.

Note, however, that the headings were also kept in the table's body — we'll change this in the next exercise.

Instructions

1. Enclose rows 2, 3, 4, 5, 6, and 7 of the table in a `<tbody>` element.

Table Head

In the last exercise, the table's headings were kept inside of the table's body. When a table's body is sectioned off, however, it also makes sense to section off the table's column headings using the `<thead>` element.

```
<table>

  <thead>

    <tr>

      <th></th>

      <th scope="col">Saturday</th>

      <th scope="col">Sunday</th>

    </tr>

  </thead>

  <tbody>
```



```

<tr>

  <th scope="row">Morning</th>

  <td rowspan="2">Work</td>

  <td rowspan="3">Relax</td>

</tr>

<tr>

  <th scope="row">Afternoon</th>

</tr>

<tr>

  <th scope="row">Evening</th>

  <td>Dinner</td>

</tr>

</tbody>

</table>

```

In the example above, the only new element is `<thead>`. The table headings are contained inside of this element. Note that the table's head still requires a row in order to contain the table headings.

Additionally, only the **column** headings go under the `<thead>` element. We can use the `scope` attribute on `<th>` elements to indicate whether a `<th>` element is being used as a "row" heading or a "col" heading.

Instructions

1. Enclose the first row of the table in a `<thead>` element.

Table Footer

The bottom part of a long table can also be sectioned off using the `<tfoot>` element.

```
<table>

  <thead>

    <tr>

      <th>Quarter</th>

      <th>Revenue</th>

      <th>Costs</th>

    </tr>

  </thead>

  <tbody>

    <tr>

      <th>Q1</th>

      <td>$10M</td>

      <td>$7.5M</td>

    </tr>

    <tr>

      <th>Q2</th>

      <td>$12M</td>
```

```

    </tr>

</tbody>

<tfoot>

    <tr>

        <th>Total</th>

        <td>$22M</td>

        <td>$12.5M</td>

    </tr>

</tfoot>

</table>

```

In the example above, the footer contains the totals of the data in the table. Footers are often used to contain sums, differences, and other data results.

Instructions

1. Add a table footer at the bottom of the table using the `<tfoot>` element. Inside of the footer, add the following data:

```

    <td>Total</td>

    <td>28</td>

```

Styling with CSS

Tables, by default, are very bland. They have no borders, the font color is black, and the typeface is the same type used for other HTML elements.

CSS, or Cascading Style Sheets, is a language that web developers use to style the HTML content on a web page. You can use CSS to style tables. Specifically, you can style the various aspects mentioned above.

```
table, th, td {  
  border: 1px solid black;  
  font-family: Arial, sans-serif;  
  text-align: center;  
}
```

The code in the example above demonstrates just some of the various table aspects you can style using CSS properties.

Instructions

1. We've included a **.css** file containing instructions for styling the HTML content in the **index.html** file. In **style.css**, set the `font-size` of all table headings (`th`) and table data (`td`) to 18 pixels (18px).

```
<!DOCTYPE html>  
  
<html>  
  
<head>  
  
  <title>Ship To It - Company Packing List</title>  
  
  <link href="https://fonts.googleapis.com/css?family=Lato:100,300,400,700|Luckiest+Guy|Oxygen:300,400" rel="stylesheet">  
  
  <link href="style.css" type="text/css" rel="stylesheet">  
  
</head>  
  
<body>  
  
  <ul class="navigation">
```

```
<li></li>  
  
<li class="active">Action List</li>  
  
<li>Profiles</li>  
  
<li>Settings</li>  
  
</ul>  
  
<div class="search">Search the table</div>  
  
<table>  
  
<thead>  
  
<tr>  
  
<th>Company Name</th>  
  
<th>Number of Items to Ship</th>  
  
<th>Next Action</th>  
  
</tr>  
  
</thead>  
  
<tbody>  
  
<tr>  
  
<td colspan="2">Adam's Greenworks</td>
```

```
<td>14</td>
```

```
<td>Package Items</td>
```

```
</tr>
```

```
<tr>
```

```
<td rowspan="2">Davie's Burgers</td>
```

```
<td>2</td>
```

```
<td>Send Invoice</td>
```

```
</tr>
```

```
<tr>
```

```
<td>Baker's Bike Shop</td>
```

```
<td>3</td>
```

```
<td>Send Invoice</td>
```

```
</tr>
```

```
<tr>
```

```
<td>Miss Sally's Southern</td>
```

```
<td>4</td>
```

```
<td>Ship</td>
```

```
</tr>
```

```
<tr>
```

```
<td>Summit Resort Rentals</td>
```

```
<td>4</td>

<td>Ship</td>

</tr>

<tr>

<td>Strike Fitness</td>

<td>1</td>

<td>Enter Order</td>

</tr>

</tbody>

<tfoot>

<tr>

<td>Total</td>

<td>28</td>

</tr>

</tfoot>

</table>

</body>

</html>
```

- The `<table>` element creates a table.
- The `<tr>` element adds rows to a table.
- To add data to a row, you can use the `<td>` element.
- Table headings clarify the meaning of data. Headings are added with the `<th>` element.
- Table data can span columns using the `colspan` attribute.
- Table data can span rows using the `rowspan` attribute.
- Tables can be split into three main sections: a head, a body, and a footer.
- A table's head is created with the `<thead>` element.
- A table's body is created with the `<tbody>` element.
- A table's footer is created with the `<tfoot>` element.

All the CSS properties you learned about in this course can be applied to tables and their data.

Semantic HTML

Introduction to Semantic HTML

When building web pages, we use a combination of non-semantic HTML and *Semantic HTML*. The word semantic means “relating to meaning,” so semantic elements provide information about the content between the opening and closing tags.

By using Semantic HTML, we select HTML elements based on their meaning, not on how they are presented. Elements such as `<div>` and `<span>` are not semantic elements since they provide no context as to what is inside of those tags.

For example, instead of using a `<div>` element to contain our header information, we could use a `<header>` element, which is used as a heading section. By using a `<header>`

tag instead of a `<div>`, we provide context as to what information is inside of the opening and closing tag.

Why use Semantic HTML?

Accessibility: Semantic HTML makes webpages accessible for mobile devices and for people with disabilities as well. This is because screen readers and browsers are able to interpret the code better.

SEO: It improves the website SEO, or *Search Engine Optimization*, which is the process of increasing the number of people that visit your webpage. With better SEO, search engines are better able to identify the content of your website and weight the most important content appropriately.

Easy to Understand: Semantic HTML also makes the website's source code easier to read for other web developers.

To better understand this, you can think of comparing non-semantic HTML to going into a store with no signs on the aisles. Since the aisles aren't labeled, you don't know what products are in those aisles. However, stores that do have signs for each aisle make it a lot easier to find the items you need, just like Semantic HTML.

Instructions

Analyze the diagram and take note of key differences between non-semantic and semantic code.

Header and Nav

Let's take a look at some semantic elements that assist in the structure of a web page.

A `<header>` is a container usually for either navigational links or introductory content containing `<h1>` to `<h6>` headings.

The example below shows `<header>` in action:

```
<header>
```

```
<h1>

  Everything you need to know about pizza!

</h1>

</header>
```

Copy to Clipboard

This can be compared to the code below which uses a `<div>` tag instead of a `<header>` tag:

```
<div id="header">

  <h1>

    Everything you need to know about pizza!

  </h1>

</div>
```

Copy to Clipboard

By using a `<header>` tag, our code becomes easier to read. It is much easier to identify what is inside of the `<h1>`'s parent tags, as opposed to a `<div>` tag which would provide no details as to what was inside of the tag.

A `<nav>` is used to define a block of navigation links such as menus and tables of contents. It is important to note that `<nav>` can be used inside of the `<header>` element but can also be used on its own.

Let's take a look at the example below:

```
<header>

  <nav>

    <ul>

      <li><a href="#home">Home</a></li>

      <li><a href="#about">About</a></li>

    </ul>

  </nav>

</header>
```

By using `<nav>` as a way to label our navigation links, it will be easier for not only us, but also for web browsers and screen readers to read the code.

Now that we've learned about the `<header>` and `<nav>` elements let's add them into our code!

Instructions

1. In the code editor, find the `<div id="header">` tag and change it to `<header>`.

Note: When changing an opening tag, you must find the corresponding closing tag and change that as well. If you don't, you'll see some red in your code to indicate the error.

2. Now, find the `<div id="nav">` tag and change it to `<nav>`.

Main and Footer

Two more structural elements are `<main>` and `<footer>`. These elements along with `<nav>` and `<header>` help describe where an element is located based on conventional web development standards.

The element `<main>` is used to encapsulate the dominant content within a webpage. This tag is separate from the `<footer>` and the `<nav>` of a web page since these elements don't contain the principal content. By using `<main>` as opposed to a `<div>` element, screen readers and web browsers are better able to identify that whatever is inside of the tag is the bulk of the content.

So how does `<main>` look when incorporated into our code? That's a great question.

```
<main>

<header>

  <h1>Types of Sports</h1>

</header>

<article>

  <h3>Baseball</h3>

  <p>

    The first game of baseball was played in Cooperstown, New York in the summer of 1839.

  </p>

</article>

</main>
```

As we see above, `<main>` contains an `<article>` and `<header>` tag with child elements that hold the most important information related to the page.

The content at the bottom of the subject information is known as the *footer*, indicated by the `<footer>` element. The footer contains information such as:

- Contact information

- Copyright information

- Terms of use

- Site Map

- Reference to top of page links

For example:

```
<footer>

<p>Email me at Codey@Codecademy.com</p>

</footer>
```

In the example above, the footer is used to contain contact information. The `<footer>` tag is separate from the `<main>` element and typically located at the bottom of the content.

Instructions

1. In the code editor, find the `<div id="main">` tag and change it to `<main>`.
2. Now, find the `<div id="footer">` tag and change it to `<footer>`.

Article and Section

Now that we covered the body of Semantic HTML, let's focus on what can go in the body. The two elements we're going to focus on now are `<section>` and `<article>`.

`<section>` defines elements in a document, such as chapters, headings, or any other area of the document with the same theme. For example, content with the same theme such as articles about cricket can go under a single `<section>`. A website's home page could be split into sections for the introduction, news items, and contact information.

Here is an example of how to use `<section>`:

```
<section>

  <h2>Fun Facts About Cricket</h2>

</section>
```

In the code above we created a `<section>` element to encapsulate the code. In `<section>` we added a `<h2>` element as a heading.

The `<article>` element holds content that makes sense on its own. `<article>` can hold content such as articles, blogs, comments, magazines, etc. An `<article>` tag would help someone using a screen reader understand where the article content (that might contain a combination of text, images, audio, etc.) begins and ends.

Here is an example of how to use `<article>`:

```
<section>

  <h2>Fun Facts About Cricket</h2>

  <article>

    <p>A single match of cricket can last up to 5 days.</p>

  </article>

</section>
```

In the code above, the `<article>` element containing a fact about cricket was placed inside of the `<section>` element. It is important to note that a `<section>` element could also be placed in an `<article>` element depending on the context.

Instructions

1. In the code find and replace `<div id="section">` with `<section>` and replace the corresponding closing `</div>` with a closing `</section>`. **Note:** When removing the `<div>` tag, make sure you remove the id attached to it!

2. Now encapsulate the `<h2>` and `<p>` tag with `<article>`.

The Aside Element

The `<aside>` element is used to mark additional information that can enhance another element but isn't required in order to understand the main content. This element can be used alongside other elements such as `<article>` or `<section>`. Some common uses of the `<aside>` element are for:

Bibliographies

Endnotes

Comments

[Pull quotes](#)

Editorial sidebars

Additional information

Here's an example of `<aside>` being used alongside `<article>`:

```
<article>

  <p>The first World Series was played between Pittsburgh and
  Boston in 1903 and was a nine-game series.</p>

</article>
```

```
<aside>

  <p>

    Babe Ruth once stated, "Heroes get remembered, but legends
never die."

  </p>

</aside>
```

As shown above, the information within the `<article>` is the important content. Meanwhile the information within the `<aside>` enhances the information in `<article>` but is not required in order to understand it.

Instructions

1.Remove the `<div id="aside">` tag and replace it with `<aside>` tag. Don't forget about the closing tag!

Figure and Figcaption

With `<aside>`, we learned that we can put additional information next to a main piece of content, but what if we wanted to add an image or illustration? That is where `<figure>` and `<figcaption>` come in.

`<figure>` is an element used to encapsulate media such as an image, illustration, diagram, code snippet, etc, which is referenced in the main flow of the document.

```
<figure>

  

</figure>
```


In this code, we created a `<figure>` element so that we can encapsulate our `<img>` tag. In `<figure>` we used the `<img>` tag to insert an image onto the webpage. We used the `src` attribute within the `<img>` tag so that we can link the source of the image.

It's possible to add a caption to the image by using `<figcaption>`.

`<figcaption>` is an element used to describe the media in the `<figure>` tag. Usually, `<figcaption>` will go inside `<figure>`. This is different than using a `<p>` element to describe the content; if we decide to change the location of `<figure>`, the paragraph tag may get displaced from the figure while a `<figcaption>` will move with the figure. This is useful for grouping an image with a caption.

```
<figure>

  

  <figcaption>This picture shows characters from
Overwatch.</figcaption>

</figure>
```

In the example above, we added a `<figcaption>` into the `<figure>` element to describe the image from the previous example. This helps group the `<figure>` content with the `<figcaption>` content.

While the content in `<figure>` is related to the main flow of the document, its position is independent. This means that you can remove it or move it somewhere else without affecting the flow of the document.

Instructions

1. Create an opening and closing `<figure>` tag under the closing `</section>` tag.

2. Add an image by using the `<img>` tag within `<figure>`. Use the following URL as the source for the image:

`https://content.codecademy.com/courses/SemanticHTML/dogimage.jpeg`

3. Create a `<figcaption>` under `<img>` and describe the image.

Audio and Attributes

Now that we learned about text-based content, let us dig into `<audio>`! Surely everyone needs `<audio>`—how else would you listen to your Korean hip hop?

The `<audio>` element is used to embed audio content into a document. Like `<video>`, `<audio>` uses `src` to link the audio source.

```
<audio>

  <source src="iAmAnAudioFile.mp3" type="audio/mp3">

</audio>
```

In this example, we created an `<audio>` element. Then we created a `<source>` element to encapsulate our audio link. In this case, `iAmAnAudioFile.mp3` is our audio file. Then we specified the type by using `type` and named what kind of audio it is. Although not always necessary, it's recommended that we state the type of audio as it helps the browser identify it more easily and determine if that type of audio file is supported by the browser.

We linked our audio file into the browser but now we need to give it controls. This is where *attributes* come in. [Attributes](#) provide additional information about an element.

Attributes allow us to do many different things to our audio file. There are many attributes for `<audio>` but today we're going to be focusing on `controls` and `src`.

`controls`: automatically displays the audio controls into the browser such as play and mute.

`src`: specifies the URL of the audio file.

As you might have noticed, we already used the `src` attribute in the example code above. Most attributes go in the opening tag of `<audio>`. For example, here's how we could add both autoplay functionality and audio controls:

```
<audio autoplay controls>
```

You can find other attributes here: [Useful attributes](#).

Instructions

1. In the code editor, create an `<audio>` tag with the attribute `controls` directly under the commented line.

2. Add a source for the audio inside the `<audio>` tag by using this code:

```
<source src="" type="audio/mp3">
```

Use the following URL as the source for the audio:

<https://content.codecademy.com/courses/SemanticHTML/dogBarking.mp3>

Video and Embed

As demonstrated in the previous exercise, media content can be a useful addition to a website. By using a `<video>` element, we can add videos to our website. The `<video>` element makes it clear that a developer is attempting to display a video to the user.

Some attributes that can alter a video playback include:

`controls`: When added in, a play/pause button will be added onto the video along with volume control and a fullscreen option.

`autoplay`: The attribute which results in a video automatically playing as soon as the page is loaded.

`loop`: This attribute results in the video continuously playing on repeat.

Below is an example of `<video>` being used with the `controls` attribute:

```
<video src="coding.mp4" controls>Video not supported</video>
```

In the code above, a video file named `coding.mp4` is being played. The “Video not supported” will only show up if the browser is unable to display the video.

Another tag that can be used to incorporate media content into a page is the `<embed>` tag, which can embed any media content including videos, audio files, and gifs from an external source. This means that websites that have an embed button have some form of media content that can be added to other websites. The `<embed>` tag is a self-closing tag, unlike the `<video>` element. Note that `<embed>` is a deprecated tag and other alternatives, such as `<video>`, `<audio>` and `<img>`, should be used in its place, but is being taught for legacy purposes.

Below we’ll take a look at `<embed>` being used in action.

```
<embed src="download.gif"/>
```

In the example above, `<embed>` is being used to add in a gif from a local file known as `download.gif`. Embed can be used to add local files as well as media content straight from some other websites.

Instructions

1. Add the following video URL with the attribute `controls` and wrap it in an opening and closing video tag.

```
https://content.codecademy.com/courses/SemanticHTML/do  
g-video.mp4
```

In between the opening and closing `<video>` tag, you can add a text that will show up if the video is unable to load. This text is helpful for screen readers as well!

2. Add the following gif into your code using the `<embed>` tag and the `src` attribute:

```
https://content.codecademy.com/courses/SemanticHTML/dog-on-beach.gif
```

```
<!DOCTYPE html>

<html>

  <head>

    <link rel="stylesheet" type="text/css" href="style.css">

  </head>

  <body>

    <header>

      <h1>Navigational Links</h1>

      <nav>

        <ul>

          <li><a href="#home">Home</a></li>

          <li><a href="#posts">Posts</a></li>

          <li><a href="#contact">Contact</a></li>

        </ul>
```

```
</nav>

</header>

<main>

  <section>

    <article>

      <h2>Facts About Dogs</h2>

      <p>

        Dogs have a sense of time. It's been proven that they
know the difference between a hour and five. If conditioned to,
they can predict future events, such as regular walk times.

      </p>

    </article>

    <aside>

      <p>A study was conducted on dogs being away from their
owners for varying hours and the studies show that dogs who were
away from their owners the longest showed the greatest amount of
affection!

      </p>

    </aside>

  </section>
```

```
<figure>

    

    <figcaption>A cute dog.</figcaption>

</figure>

<audio controls>

    <source
src="https://content.codecademy.com/courses/SemanticHTML/dogBarking.mp3" type="audio/mp3">

</audio>

<!-- Create <video> and <embed> tag here -->

<video
src="https://content.codecademy.com/courses/SemanticHTML/dog-video.mp4" controls>Video not supported</video>

<embed
src="https://content.codecademy.com/courses/SemanticHTML/dog-on-beach.gif">

</main>

<footer>

    <p>Contact me at +1 234 567 8910 </p>

</footer>
```

```
</body>  
  
</html>
```

1. Semantic HTML introduces meaning to a page through specific elements that provide context as to what is in between the tags.
2. Semantic HTML is a modern standard and makes a website accessible for people who use screen readers to translate the webpage and improves your website's SEO.
3. `<header>`, `<nav>`, `<main>` and `<footer>` create the basic structure of the webpage.
4. `<section>` defines elements in a document, such as chapters, headings, or any other area of the document with the same theme.
5. `<article>` holds content that makes sense on its own such as articles, blogs, comments, etc.
6. `<aside>` contains information that is related to the main content, but not required in order to understand the dominant information.
7. `<figure>` encapsulates all types of media.
8. `<figcaption>` is used to describe the media in `<figure>`.
9. `<video>`, `<embed>`, and `<audio>` elements are used for media files.

